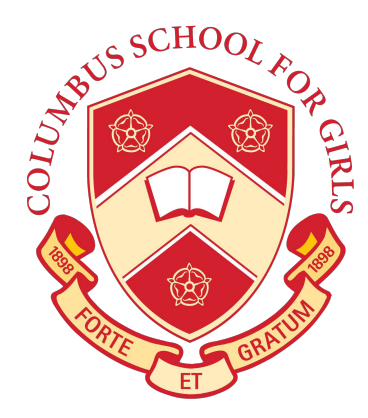
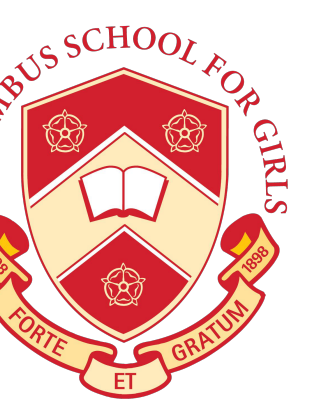




CAN Protocol and Controlling an Instrument Panel Cluster



Lina Jeffers

Introduction

This summer I worked at OSU's CAR with their Buckeye Autodrive team. The team is a part of the SAE AutoDrive Challenge, a 4 year competition of 10 universities to develop a level 4 autonomous vehicle that can navigate urban driving courses. The vehicle is expected to be able to recognize and obey traffic signs and signals and avoid obstacles completely autonomously. This was the first year of the the AutoDrive Challenge II, focused on creating a framework to build off of in the future years and judging perception systems ability to quickly and accurately identify and report different signs, objects, and the status of traffic lights. During my time at CAR, I worked to learn about, understand, and put into practice the basics of CAN protocol.

What is CAN?

CAN is a message based protocol for communicating between different nodes, or ECU's (electric control units) in a vehicle. It functions like the nervous system of a CAR, acting as a way for information to be sent between different systems like the airbags, door control, or the speedometer so that they can communicate crucial information quickly. CAN allows all of these nodes to send and receive messages to each other, without requiring direct connections between each.

How does it work?

Physically, a can bus consists of two wires, CAN high and CAN low, connected with 120 Ohm resistors on each end. Each node is connected to each line in parallel, and signals are read from the difference in voltage of the two signals (allows for robust system). Each message is encoded by one node on the bus, identified through its given ID number, and decoded by the receiving node through a shared "dbc file".

All connected nodes can send messages to the bus, and can see all messages being transmitted. Each node can then choose to either ignore or receive messages if they are relevant.

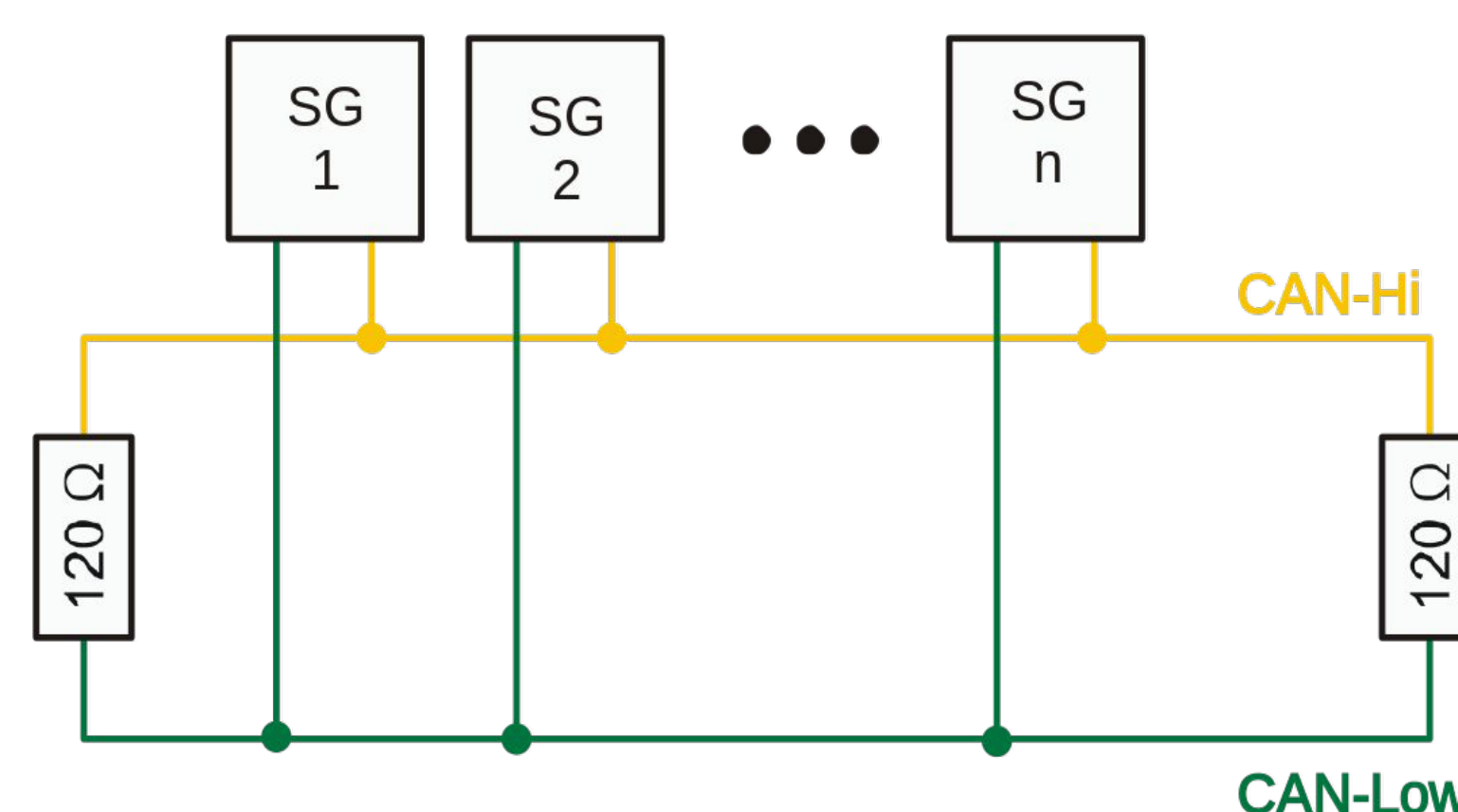


Figure 1. Example CAN bus setup

Project Overview

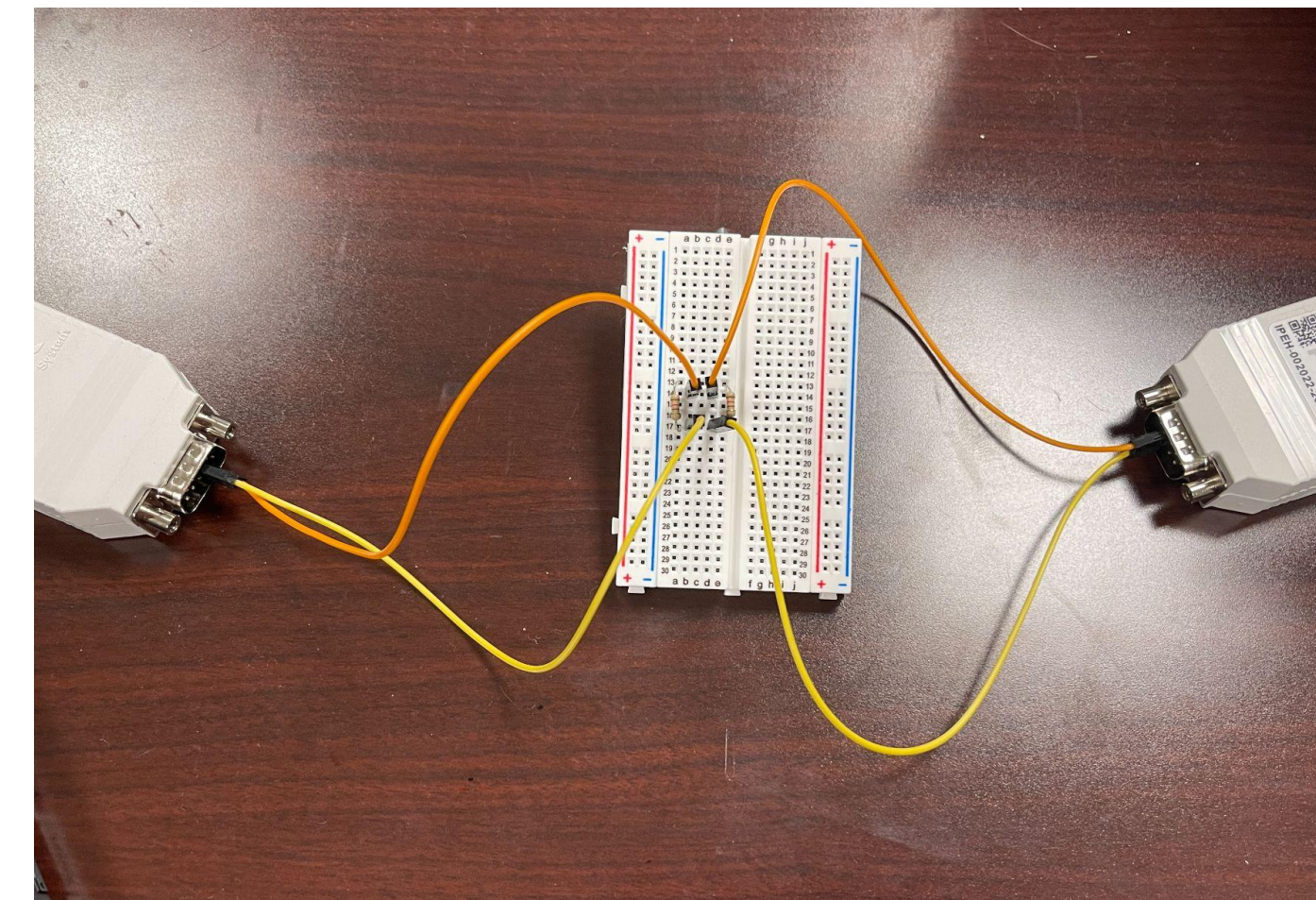


Figure 2. Initial basic CAN setup.

Next, I used these basic skills to prepare my final project, using CAN messages to control an instrument control panel (or ipc) from an actual car.

I was creating a system that would allow the user to adjust the speed and rpm dials on it with two analog potentiometers or nobs. To do this, I would connect two potentiometers to a NXP microcontroller through which I could work in Matlab and Simulink to translate the analog signals to CAN messages to then be encoded and sent to the ipc.

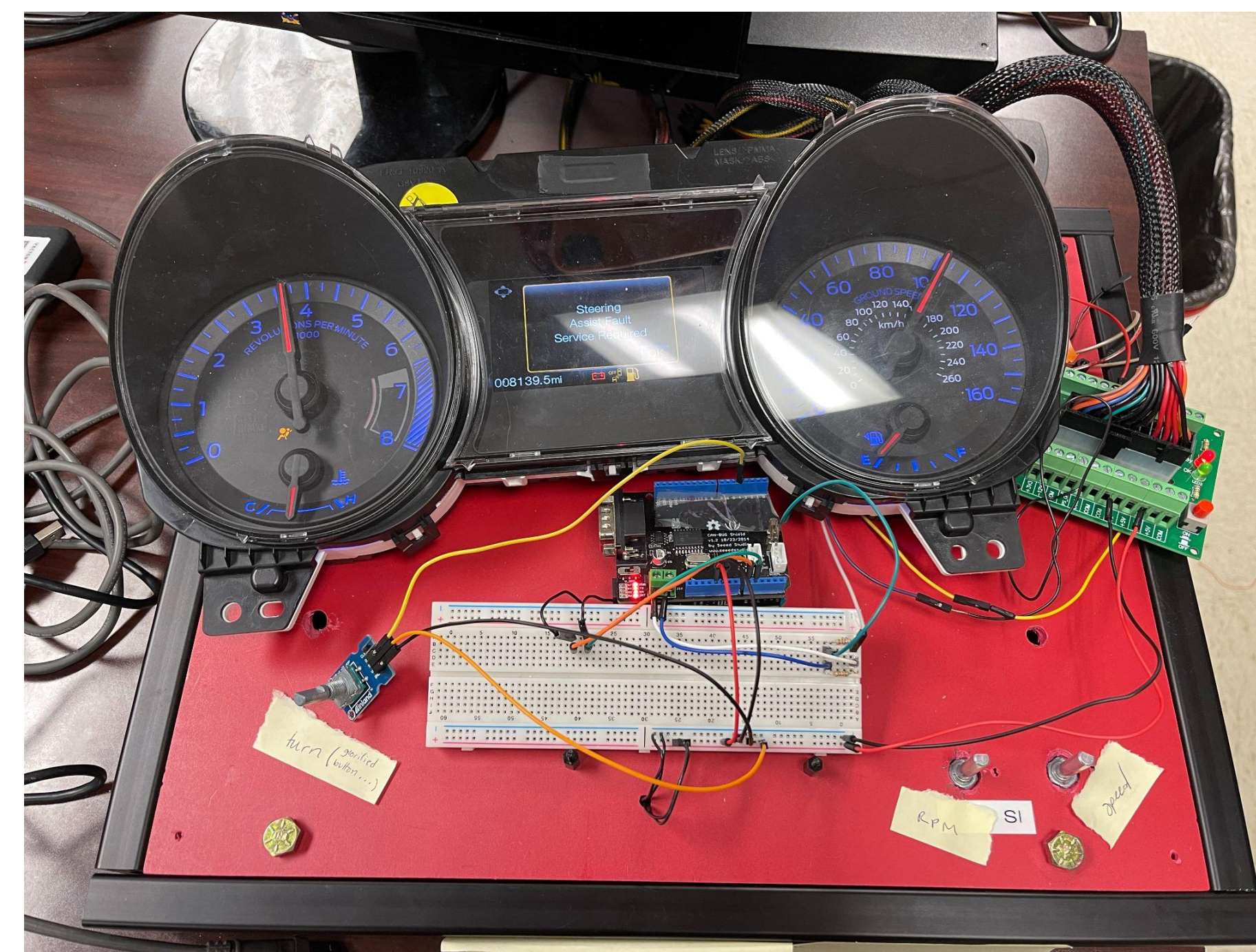


Figure 3. Final Arduino to IPC setup.

The first project I set up was just a simple setup connecting two laptops over CAN so I could send and monitor messages between them. I was then able to practice several different ways that you can send and receive can messages, such as through specific software, python scripts, or simply a few commands in a linux terminal.

Challenges and Adjustments

The biggest challenge I encountered was finding the correct messages needed to actually work with the IPC, since I didn't have the actual mustang dbc file to match the one used by the ipc, guessing the message ID that needed to be sent would be nearly impossible. I also needed to find not just the messages for the speed and rpm gauges, but also a specific sequence of other messages to keep the ipc on and running without an actual vehicle. This information and the dbc files are not publicly available, but I was able to learn a lot about the process of car hacking and how you would and could find the ID's for your own car by monitoring the bus while driving around and changing various conditions. I was however able to get the dbc file from the driving dynamics lab at CAR which they are using for the driving simulator they are working on. Additionally, the nxp board I had been given had some problems connecting to the CAN bus, so I switched to an arduino with a CAN bus shield. Because of this change I also could not use Matlab's built in encoding functions and had to manually encode the messages I needed through the arduino IDE.

Results

The final setup pictured in Figure 3 consisted of a Mustang IPC connected over CAN to an Arduino with a CAN bus and a power source. The Arduino was connected to three analog potentiometers, one that controlled the speedometer, one for the tachometer, and a third that would control the turn signal indicator. The potentiometers and arduino are simply connected over a breadboard to each other and the power source.

While not included in the original plan, I was able to incorporate the turn signal control as I discovered the needed CAN messages while determining which ID's controlled the other components of the IPC. The Arduino converts the analog signal from each of the potentiometers into the correct hexadecimal values, and then encodes them to match the required range known by the dbc file on the IPC.

Takeaways

Through my work on this project, I learned not only what CAN is and how it is used on the Buckeye autoDrive team, but also how important it is to our everyday lives. Besides CAN I also learned more about how to find and filter information and utilize available resources, especially at a place like CAR and gained a more in depth understanding of how electrical signals allow us to transfer information. While not specifically related to my project, I also learned some basic information in other areas. This includes more of the basics of how cars work internally and how one could hack into their own car, a much better understanding of binary and hexadecimal number bases, familiarity with different microcontrollers and their respective software IDE's, and how to dual boot and work in a Linux OS on a flash drive.

Literature Cited

Buckeye AutoDrive Places Second Overall in Inaugural Competition. (2022, June 16). OSU.edu.

Retrieved April 9, 2023, from <https://car.osu.edu/news/2022/06/buckeye-autodrive-places-second-overall-inaugural-competition?fbclid=IwAR3jEk0B0FJouZtuIaONUekFCO2sJKejd9zyrCTGqL78yqcoFboiACNHwuA>

[CAN Diagram]. (2011, June 14). Wikipedia. https://en.wikipedia.org/wiki/File:CAN-Bus_Elektrische_Zweidrahtleitung.svg#filelinks

CAN Bus Protocol: The Ultimate Guide (2023). (2023, January 29). AutoPi.

Retrieved April 9, 2023, from <https://www.autopi.io/blog/can-bus-explained/>

BUCKEYE AUTODRIVE & CAN PROTOCOL

Lina Jeffers



SAE AutoDrive Challenge II

- 4 year competition
- 10 Universities
- Develop level 4 autonomous vehicle
- Navigate simulated urban roads
 - Obey signs
 - Avoid obstacles



Buckeye AutoDrive Team



- Placed 2nd in 2022
- Year 1 Goals
 - Perception and sensors
 - Cameras/LiDaR
 - Accurately and quickly identify and report data

Understanding

Controller Area Networks (CAN)

WHAT IS CAN PROTOCOL?

- Message-based protocol
 - Communication between nodes/ECU's
 - Airbags, door control, speedometer, etc.
 - Priority based
 - Direct connections not required
- Used in AutoDrive to monitor and send perception data



WHY CAN?



Lower Cost



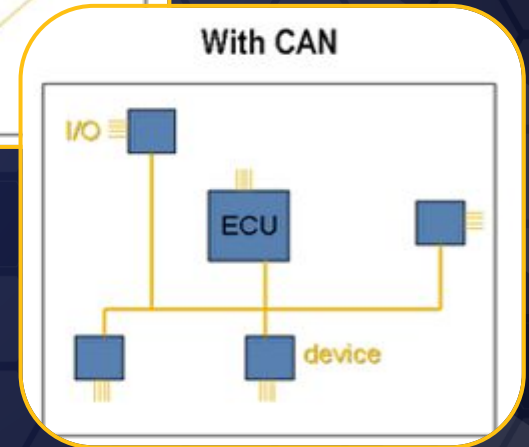
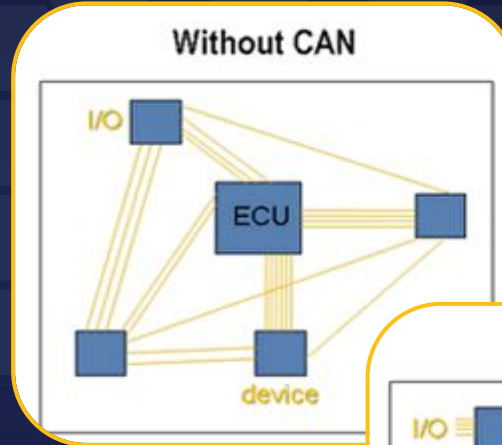
Robust



Efficient

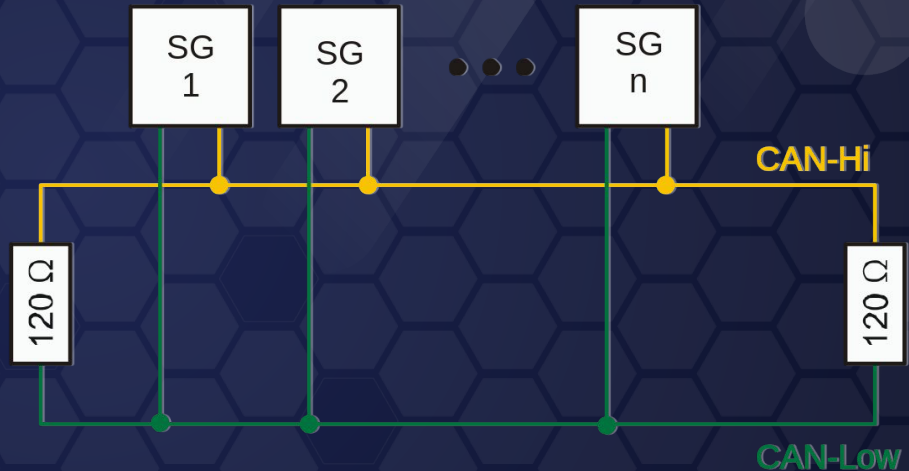


Centralized and Flexible



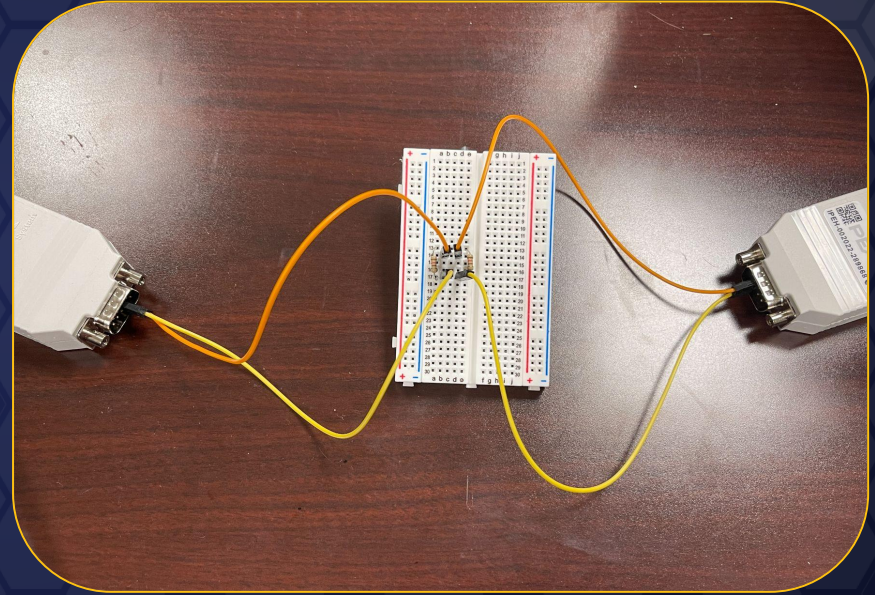
WHAT IS A CAN BUS?

- Two wires
 - Hi and Low voltage
 - Nodes connected to both wires in parallel
 - 120 Ohm resistors on each end
 - Differential signaling
- Messages seen by all nodes



Project 1

- Two connected laptops (nodes)
- Each can receive and transmit messages
 - Many ways to send/receive messages



```
can0 316 [8] 00 00 00 00 00 00 00 00
can0 317 [8] 03 FE 00 00 00 00 00 00
can0 318 [8] 00 00 00 00 00 00 00 00
can0 346 [8] 00 00 00 03 03 00 C0 00
can0 348 [8] 00 00 00 00 00 00 00 00
can0 3E0 [8] 00 00 00 00 80 00 00 00
can0 3FB [8] 00 00 00 00 04 00 00 00
can0 430 [8] 04 00 33 2B 00 00 64 00
can0 434 [8] 40 00 10 00 28 FF F0 00
can0 45A [8] C7 00 00 00 00 00 00 00
can0 346 [8] 00 00 00 03 03 00 C0 00
can0 348 [8] 00 00 00 00 00 00 00 00
can0 430 [8] 04 00 33 2B 00 00 64 00
can0 434 [8] 40 00 10 00 28 FF F0 00
can0 287 [8] 00 00 00 00 00 00 00 00
can0 30A [8] C7 00 00 00 00 00 00 00
can0 346 [8] 00 00 00 03 03 00 C0 00
can0 348 [8] 00 00 00 00 00 00 00 00
```

Data Length

can0 430 [8] 04 00 33 2B 00 00 64 00

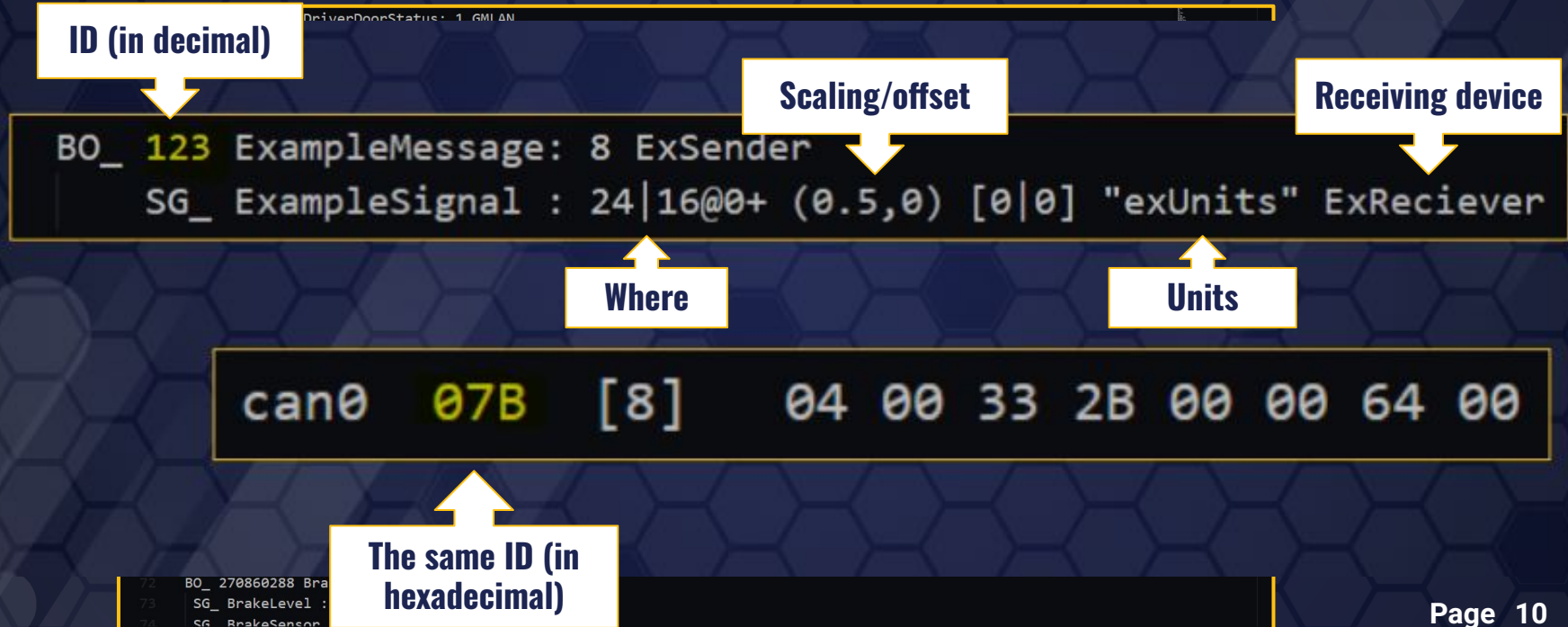
CAN ID

Data Payload

CAN MESSAGES

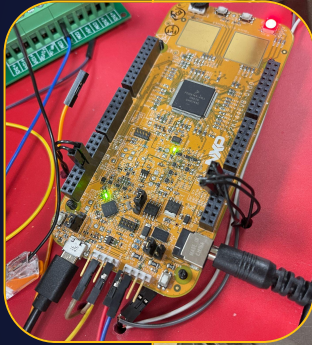
DBC FILES

TRANSLATING CAN FRAMES

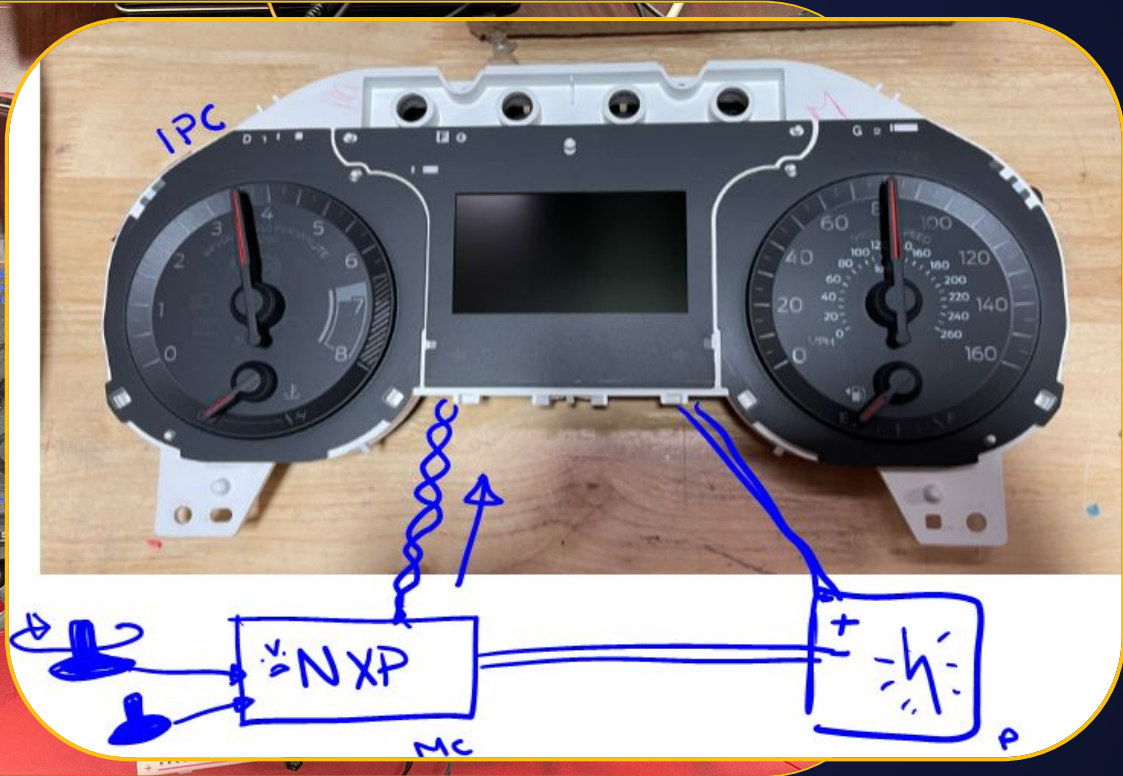
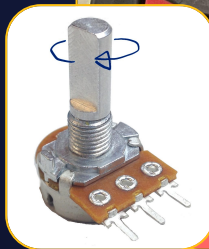


Project 2: Instrument Panel Cluster

NXP microcontroller



Potentiometer



Challenges: Finding CAN ID's

- Specific sequence unknown
 - No actual car
 - Not publicly available
- Car hacking
- Saved by the Driving dynamics lab

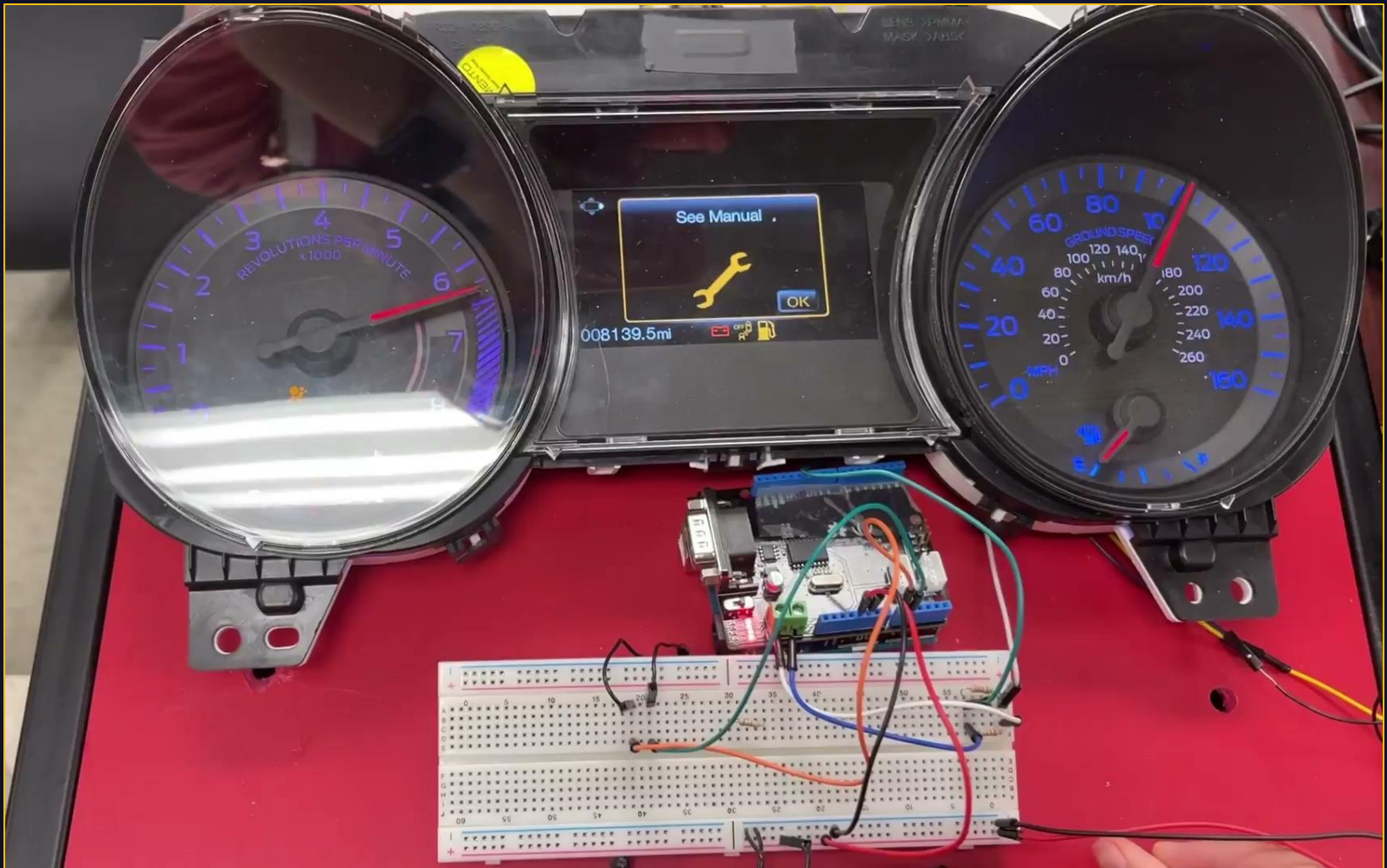


Writing **Arduino Code**

- Reading and scaling potentiometers
- Manually encoding
 - Decimal to Hexadecimal

```
void DecToBin(int dec) { //converts decimal number to four groups of 4 binary bits  
  
unsigned char toHex(String digits) { //finds the corresponding hex char for each group of 4 bits  
  
unsigned char* hex_decode(const char *in, size_t len, unsigned char *out) //converts two characters into one hex byte  
{
```

- Instead of library or toolbox with built in functions to encode
- Continuously sending messages



TAKEAWAYS

1. CAN Protocol
2. Ask specific questions
 - a. Filtering advice
3. Online resources
4. Adjacent topics
 - a. Car hacking
 - b. Microcontrollers and sensors
 - c. Linux



SPECIAL THANKS

Dr. Qadeer
Ahmed

Shreya Pal

Pradeep
Oruganti

Dr. Leonard
Brillson

Dr. Kevin
Sweeney

The OSU
EcoCar Team

SOURCES

Buckeye AutoDrive Places Second Overall in Inaugural Competition. (2022, June 16). The Ohio State University. Retrieved July 23, 2022, from <https://car.osu.edu/news/2022/06/buckeye-autodrive-places-second-overall-inaugural-competition?fbclid=IwAR3jEk0B0FJouZtulaONUekfCO2sJKejd9zyrCTGgLT78yqcoFboiACNHwuA>

CAN Bus Protocol: The Ultimate Guide (2022). (2022, March). AutoPi. Retrieved July 22, 2022, from <https://www.autopi.io/blog/can-bus-explained/>

Falch, M. (2022, April). *CAN DBC File Explained - A Simple Intro* [+Editor Playground]. CSS Electronics. Retrieved July 22, 2022, from <https://www.csselectronics.com/pages/can-dbc-file-database-intro>

Potentiometer [Photograph]. (2016, July 11). Build Electronic Circuits. <https://www.build-electronic-circuits.com/potentiometer/>

Derks, F. (2018, October). Collision Avoidance Systems [Image]. Medium. <https://medium.com/@theteamorcad/how-do-collision-avoidance-systems-work-eee02adc745>

Stefan-Xp. (2008, February). [CAN-Bus Topology (Electronic Two Wire Connection)] [Infographic]. https://en.wikipedia.org/wiki/File:CAN-Bus_Elektrische_Zweidrahtleitung.svg#filelinks

[With and without CAN]. (n.d.). Chipkin. <https://store.chipkin.com/articles/can-bus-protocol-10-minute-lesson>

This presentation template was created by Slidesgo, and it includes icons by Flaticon, and infographics & images by Freepik

THANK YOU

Any Questions?